

Extra 6 · PL/SQL Exception Handling

Why this matters in interviews. "How do you handle errors in PL/SQL?", "What is `NO_DATA_FOUND`?", "Difference between `RAISE` and `RAISE_APPLICATION_ERROR`?", "What does `WHEN OTHERS` do and why is it dangerous?", "What is `PRAGMA EXCEPTION_INIT`?" — exception handling is asked in every PL/SQL round, and it is the natural follow-up to the block, cursor and trigger chapters (12–15).

In this chapter: errors vs exceptions · the `EXCEPTION` section · predefined exceptions · `SQLCODE` / `SQLERRM` · user-defined exceptions and `RAISE` · `PRAGMA EXCEPTION_INIT` · `RAISE_APPLICATION_ERROR` · propagation rules · re-raising · transactions and exceptions · logging · best practices.

Verification note. These are PL/SQL blocks, and Oracle is not available where these notes were built, so — like Chapters 12–15 — every "Output" shown was worked out by hand against the sample tables rather than executed. Run each block in SQL Developer once (`SET SERVEROUTPUT ON` first) to see it for yourself.

E6.1 Two kinds of errors

	Compile-time	Run-time (exceptions)
Found	when the block/subprogram is compiled	while it is running
Examples	typo in a name, missing <code>;</code> , wrong type (<code>PLS-00201: identifier must be declared</code>)	no row found, divide by zero, duplicate key, your own rule violated
Handled by	you fix the code (<code>SHOW ERRORS</code> lists them)	an exception handler in the <code>EXCEPTION</code> section

An **exception** is a run-time error *event*. When one occurs, normal execution stops and control **jumps** to the exception handler; if there is no matching handler the exception travels outward.

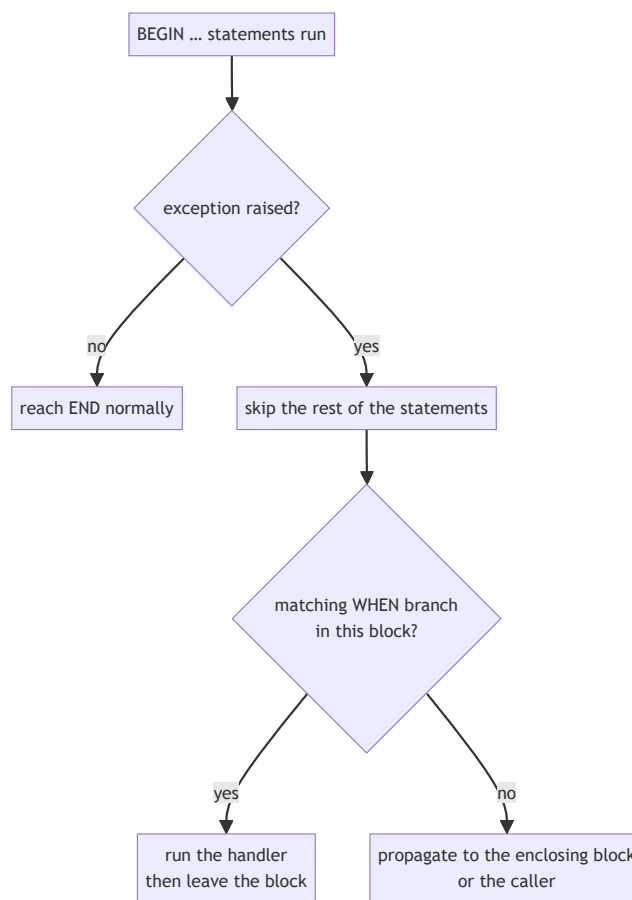
E6.2 The `EXCEPTION` section

It is the optional last part of every block (Chapter 12):

```

DECLARE
    -- variables
BEGIN
    -- executable statements
EXCEPTION
    WHEN exception_name_1 THEN
        -- handle it
    WHEN exception_name_2 OR exception_name_3 THEN
        -- handle either
    WHEN OTHERS THEN
        -- handle anything not matched above
END;
/

```



Three rules that answer many questions:

1. After a handler runs, the block **ends**. Execution does **not** resume at the statement after the one that failed. (It continues after the *enclosing* block's call to it, if any.)
2. `WHEN OTHERS` must be the **last** branch — it catches everything else.
3. Only **one** handler runs.

E6.3 Predefined exceptions

Oracle gives names to the most common errors. You never declare these; they are already in scope (package `STANDARD`).

Exception	Oracle error	Raised when
<code>NO_DATA_FOUND</code>	ORA-01403	<code>SELECT ... INTO</code> returns no row
<code>TOO_MANY_ROWS</code>	ORA-01422	<code>SELECT ... INTO</code> returns more than one row
<code>ZERO_DIVIDE</code>	ORA-01476	division by zero
<code>DUP_VAL_ON_INDEX</code>	ORA-00001	insert/update breaks a <code>PRIMARY KEY</code> / <code>UNIQUE</code> constraint
<code>INVALID_NUMBER</code>	ORA-01722	a string can't be converted to a number in a SQL statement
<code>VALUE_ERROR</code>	ORA-06502	conversion, truncation or size error in PL/SQL (e.g. <code>'abc'</code> into a <code>NUMBER</code> , 5 characters into a <code>VARCHAR2(3)</code>)
<code>CURSOR_ALREADY_OPEN</code>	ORA-06511	opening a cursor that is already open
<code>INVALID_CURSOR</code>	ORA-01001	fetching from / closing a cursor that isn't open
<code>LOGIN_DENIED</code>	ORA-01017	wrong username/password
<code>CASE_NOT_FOUND</code>	ORA-06592	a <code>CASE</code> statement with no matching branch and no ELSE
<code>COLLECTION_IS_NULL</code>	ORA-06531	using a collection (e.g. <code>VARRAY</code>) that was never initialised
<code>SUBSCRIPT_BEYOND_COUNT</code>	ORA-06533	index larger than the collection's size
<code>SUBSCRIPT_OUTSIDE_LIMIT</code>	ORA-06532	index outside the allowed range (e.g. 0 or a negative number)
<code>ACCESS_INTO_NULL</code>	ORA-06530	assigning to an attribute of an uninitialised object
<code>PROGRAM_ERROR</code> / <code>STORAGE_ERROR</code>	ORA-06501 / ORA-06500	internal PL/SQL problem / out of memory
<code>TIMEOUT_ON_RESOURCE</code>	ORA-00051	waited too long for a resource

Two look-alikes to keep apart: `INVALID_NUMBER` is for conversions inside *SQL statements*; `VALUE_ERROR` is for conversions in *procedural* code.

`NO_DATA_FOUND` and `TOO_MANY_ROWS` — the two you will meet daily

`SELECT ... INTO` must return **exactly one** row.

```

SET SERVEROUTPUT ON
DECLARE
    v_name emp.emp_name%TYPE;
BEGIN
    SELECT emp_name INTO v_name FROM emp WHERE emp_id = 999;    -- no such employee
    DBMS_OUTPUT.PUT_LINE('Found: ' || v_name);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee with that id');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('More than one employee matched');
END;
/

```

Output:
No employee with that id

Change the query to `WHERE dept_id = 20` and the table has five such employees, so the output becomes `More than one employee matched`.

Trap. `NO_DATA_FOUND` is raised by `SELECT ... INTO` — **not** by an aggregate query without a `GROUP BY`. `SELECT MAX(salary) INTO v FROM emp WHERE emp_id = 999;` returns one row containing `NULL`, so no exception occurs. Also, an explicit-cursor `FETCH` past the last row does **not** raise it either — test `c%NOTFOUND` (Chapter 14).

ZERO_DIVIDE

```

DECLARE
    v_total NUMBER := 100;
    v_count NUMBER := 0;
    v_avg NUMBER;
BEGIN
    v_avg := v_total / v_count;
    DBMS_OUTPUT.PUT_LINE('Average = ' || v_avg);
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Cannot divide by zero - no items');
END;
/

```

Output:
Cannot divide by zero - no items

DUP_VAL_ON_INDEX

```
BEGIN
    INSERT INTO dept VALUES (10, 'Finance', 'Delhi');    -- dept_id 10 already exists
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Department 10 already exists');
END;
/
```

Output:
Department 10 already exists

Several exceptions in one handler

```
EXCEPTION
    WHEN NO_DATA_FOUND OR TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Expected exactly one row');
```

E6.4 SQLCODE and SQLERRM — what exactly happened?

Inside a handler, two functions describe the *current* exception:

Function	Returns
SQLCODE	the error number : a negative number (-1422), +100 for NO_DATA_FOUND , and +1 for a user-defined exception
SQLERRM	the message text , e.g. ORA-01422: exact fetch returns more than requested number of rows

```
DECLARE
    v_name emp.emp_name%TYPE;
BEGIN
    SELECT emp_name INTO v_name FROM emp WHERE dept_id = 20;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Code   : ' || SQLCODE);
        DBMS_OUTPUT.PUT_LINE('Message: ' || SQLERRM);
END;
/
```

Output:
Code : -1422
Message: ORA-01422: exact fetch returns more than requested number of rows

- `SQLCODE` and `SQLERRM` **cannot be used directly inside a SQL statement** — copy them into variables first (`v_code := SQLCODE;`).
- `SQLERRM` is limited to 512 bytes; `DBMS_UTILITY.FORMAT_ERROR_STACK` returns the full message.
- `SQLCODE` is **0** when there is no exception.

E6.5 WHEN OTHERS — and why it is dangerous

`WHEN OTHERS` catches every exception not named above it. It is fine as a **last-resort safety net that logs and re-raises**, and terrible as a way to silence errors.

```
-- ❌ the worst pattern in PL/SQL: the error vanishes, the caller thinks all is well
EXCEPTION
  WHEN OTHERS THEN
    NULL;
```

Every real failure — a full disk, a bug, a locked row — would be swallowed with no trace. A safer form:

```
EXCEPTION
  WHEN OTHERS THEN
    log_error(SQLCODE, SQLERRM); -- record what happened
    RAISE;                       -- and let the caller know
```

E6.6 User-defined exceptions

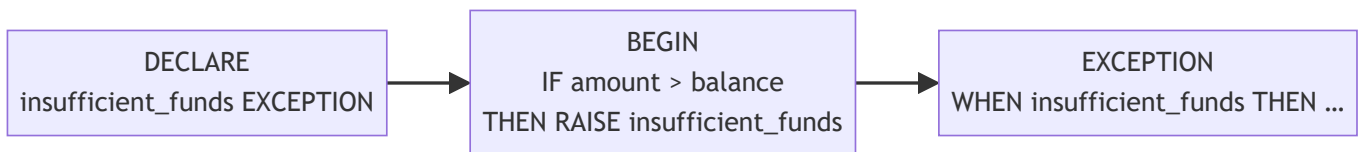
Sometimes the "error" is a **business rule**, not an Oracle error — for example, a withdrawal larger than the balance. You define a named exception, **raise it yourself** and handle it.

Three steps: **declare** (in `DECLARE`) → **raise** (`RAISE` in `BEGIN`) → **handle** (`WHEN` in `EXCEPTION`).

```
DECLARE
  v_balance      NUMBER := 500;
  v_withdraw     NUMBER := 800;
  insufficient_funds EXCEPTION;           -- 1. declare
BEGIN
  IF v_withdraw > v_balance THEN
    RAISE insufficient_funds;              -- 2. raise
  END IF;
  v_balance := v_balance - v_withdraw;
  DBMS_OUTPUT.PUT_LINE('New balance: ' || v_balance);
EXCEPTION
  WHEN insufficient_funds THEN            -- 3. handle
    DBMS_OUTPUT.PUT_LINE('Insufficient funds: balance ' || v_balance ||
                        ', requested ' || v_withdraw);
END;
/
```

Output:

Insufficient funds: balance 500, requested 800



- A user-defined exception has **no error number** of its own (its `SQLCODE` is `+1`, its `SQLERRM` is `User-Defined Exception`) unless you attach one — next section.
- Its **scope** is the block where it is declared (and blocks nested inside). To share one across programs, declare it in a **package specification** (e.g. `bank_pkg.insufficient_funds`), as Chapter 13's packages allow.

E6.7 PRAGMA EXCEPTION_INIT — naming an Oracle error

Oracle has thousands of errors but only about twenty predefined names. If you want to catch, say, **ORA-02292** (*child record found* — you tried to delete a parent row that still has children) by name, you **bind a name to the number**:

```
DECLARE
    child_records_exist EXCEPTION;
    PRAGMA EXCEPTION_INIT(child_records_exist, -2292);    -- name ↔ error number
BEGIN
    DELETE FROM dept WHERE dept_id = 20;                -- employees reference it (with an FK in place)
EXCEPTION
    WHEN child_records_exist THEN
        DBMS_OUTPUT.PUT_LINE('Cannot delete: employees still belong to this department');
END;
/
```

- A **pragma** is a compiler directive; `EXCEPTION_INIT` is processed at compile time.
- The number must be the **negative** Oracle error code (`-2292`, not `2292`) — except that `+100` (no data found) is also allowed.
- The declaration goes in the `DECLARE` section (or a package).
- Same technique works for your own `RAISE_APPLICATION_ERROR` codes (next section).

Useful error numbers to bind: `-1` (unique violation, already `DUP_VAL_ON_INDEX`), `-1400` (cannot insert NULL), `-2290` (check constraint), `-2291` (parent key not found), `-2292` (child record found), `-54` (resource busy, `NOWAIT`), `-60` (deadlock).

E6.8 RAISE_APPLICATION_ERROR

`RAISE` is for named exceptions *inside* PL/SQL. To report your own **error number and message to the caller** (a client application, or the user running the statement) use the built-in procedure:

```
RAISE_APPLICATION_ERROR(error_number, message [, keep_errors]);
```

Argument	Rule
error_number	-20000 to -20999 (reserved for applications)
message	text up to 2048 bytes
keep_errors	optional Boolean; TRUE adds this error to the existing error stack instead of replacing it

```
CREATE OR REPLACE PROCEDURE give_raise (p_emp_id IN NUMBER, p_percent IN NUMBER) IS
BEGIN
    IF p_percent NOT BETWEEN 0 AND 50 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Raise must be between 0 and 50 percent');
    END IF;

    UPDATE emp SET salary = salary * (1 + p_percent / 100) WHERE emp_id = p_emp_id;

    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'No employee with id ' || p_emp_id);
    END IF;
END;
/
```

Calling `give_raise(101, 80)` from SQL*Plus shows an error like:

```
ORA-20001: Raise must be between 0 and 50 percent
ORA-06512: at "SCOTT.GIVE_RAISE", line 4
ORA-06512: at line 1
```

(The `ORA-06512` lines are the call stack — the schema, procedure name and line numbers depend on where it ran, so yours will differ.)

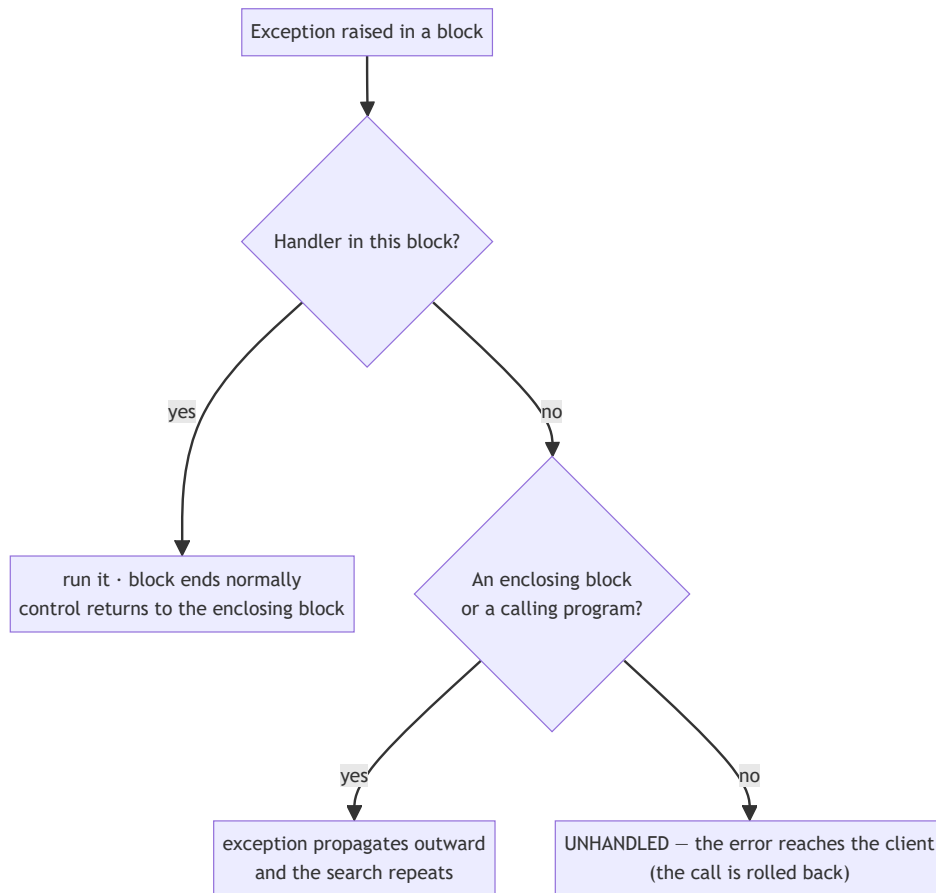
RAISE VS RAISE_APPLICATION_ERROR

	RAISE	RAISE_APPLICATION_ERROR
Used for	a named exception (predefined or user-defined)	your own code and message
Error number	that exception's own	-20000 ... -20999
Message	the exception's default (<code>User-Defined Exception</code> for yours)	any text you choose
Typical caller	another PL/SQL block	the client application / end user

In triggers. `RAISE_APPLICATION_ERROR` inside a trigger **aborts the triggering DML statement** and rolls it back — the standard way for a trigger to reject a change (Chapter 15).

E6.9 How exceptions propagate

When an exception is raised, PL/SQL looks for a handler like this:



Nested blocks — handling errors locally so processing continues

```
DECLARE
    v_name emp.emp_name%TYPE;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Start');

    BEGIN                                     -- inner block
        SELECT emp_name INTO v_name FROM emp WHERE emp_id = 999;
        DBMS_OUTPUT.PUT_LINE('Inner: ' || v_name); -- skipped
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Inner handler: no such employee');
    END;

    DBMS_OUTPUT.PUT_LINE('Outer continues'); -- this DOES run
END;
/
```

Output:
Start
Inner handler: no such employee
Outer continues

The inner block handled its own exception, ended, and the outer block carried on. **This "sub-block per risky step" pattern is how you continue a loop after a bad row:**

```
BEGIN
  FOR rec IN (SELECT emp_id FROM emp) LOOP
    BEGIN                                -- one sub-block per iteration
      process_employee(rec.emp_id);      -- may raise an exception
    EXCEPTION
      WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Skipped ' || rec.emp_id || ': ' || SQLERRM);
    END;
  END LOOP;
END;
/
```

Without the inner block, the first failure would end the whole loop.

Not handled here → handled by the *outer* block

```
BEGIN
  BEGIN
    DBMS_OUTPUT.PUT_LINE(10 / 0);        -- ZERO_DIVIDE
  EXCEPTION
    WHEN NO_DATA_FOUND THEN             -- wrong exception: no match
      DBMS_OUTPUT.PUT_LINE('inner: no data');
  END;
  DBMS_OUTPUT.PUT_LINE('never printed');
EXCEPTION
  WHEN ZERO_DIVIDE THEN
    DBMS_OUTPUT.PUT_LINE('outer caught: divide by zero');
END;
/
```

Output:
outer caught: divide by zero

Two places an exception is **not** caught by the same block

These are classic "gotcha" questions:

1. **Errors in the DECLARE section.** If initialising a variable fails, the block's own handler cannot catch it — the exception propagates to the *enclosing* block.

```
sql DECLARE v_small NUMBER(2) := 1000; -- too big: VALUE_ERROR raised HERE BEGIN NULL; EXCEPTION WHEN
VALUE_ERROR THEN -- does NOT catch it DBMS_OUTPUT.PUT_LINE('caught'); END; /
```

Result: the block fails with `ORA-06502: PL/SQL: numeric or value error: number precision too large`. To catch it, wrap this block in an outer block that has the handler.

2. **Errors inside an exception handler.** If a handler itself raises an exception, the *other* handlers of the same block do not catch it; it propagates outward.

E6.10 Re-raising

Inside a handler, a bare `RAISE;` **re-raises the same exception** to the enclosing block or caller. Use it to log and pass on:

```
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('logging: employee missing');
    RAISE; -- caller still sees ORA-01403
```

You can also raise a *different*, more meaningful exception to translate a low-level error into a business one:

```
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20010, 'Employee ' || p_emp_id || ' does not exist');
```

`RAISE;` with no name is legal **only inside an exception handler**.

E6.11 Exceptions and transactions

Situation	What happens to the data
Exception unhandled , propagates out of the outermost block to the client	the failed call is rolled back as a unit (Oracle's statement-level atomicity — an implicit savepoint is taken before the call). Work committed earlier is safe. <i>(The exact outcome is decided by the calling environment, so don't rely on it for important logic — roll back explicitly.)</i>
Exception handled by a handler (including one caught by a <i>calling</i> PL/SQL block)	No automatic rollback. Changes made before the error stay in the transaction — you decide whether to <code>ROLLBACK</code>
<code>COMMIT</code> before the error	already permanent, cannot be undone

```

BEGIN
    UPDATE emp SET salary = salary + 1000 WHERE emp_id = 101;      -- succeeds
    UPDATE emp SET salary = salary / 0 WHERE emp_id = 102;      -- ZERO_DIVIDE
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        ROLLBACK;          -- explicitly undo the first
update
        DBMS_OUTPUT.PUT_LINE('Rolled back - nothing changed');
END;
/

```

Trap. Handling an exception does *not* undo the earlier DML. If both updates must succeed together, either let the exception propagate (the call is rolled back for you) or `ROLLBACK` (or `ROLLBACK TO` a `SAVEPOINT`) in the handler.

E6.12 Logging errors — autonomous transactions

You want to record errors in a table even when the main transaction is rolled back. An **autonomous transaction** commits independently of the caller:

```

CREATE TABLE error_log (
    logged_at  TIMESTAMP DEFAULT SYSTIMESTAMP,
    err_code   NUMBER,
    err_msg    VARCHAR2(4000),
    err_where  VARCHAR2(4000)
);

CREATE OR REPLACE PROCEDURE log_error (p_code IN NUMBER, p_msg IN VARCHAR2) IS
    PRAGMA AUTONOMOUS_TRANSACTION;          -- its own independent transaction
BEGIN
    INSERT INTO error_log (err_code, err_msg, err_where)
    VALUES (p_code, p_msg, DBMS_UTILITY.FORMAT_ERROR_BACKTRACE);
    COMMIT;                                -- commits ONLY the log row
END;
/

```

Usage:

```

EXCEPTION
    WHEN OTHERS THEN
        log_error(SQLCODE, SQLERRM);
        RAISE;

```

Diagnostic function	Gives
<code>SQLCODE</code> , <code>SQLERRM</code>	number and message of the current exception

Diagnostic function	Gives
<code>DBMS_UTILITY.FORMAT_ERROR_STACK</code>	the full error message stack
<code>DBMS_UTILITY.FORMAT_ERROR_BACKTRACE</code>	the line numbers where the exception originated (call this <i>at the top of</i> the handler, before another exception replaces it)
<code>DBMS_UTILITY.FORMAT_CALL_STACK</code>	the call stack at the moment it is called

E6.13 Related features worth a mention

Feature	Purpose
<code>FORALL ... SAVE EXCEPTIONS</code>	bulk DML that doesn't stop at the first bad row ; failures collected in <code>SQL%BULK_EXCEPTIONS</code> ; then raises ORA-24381 at the end
<code>LOG ERRORS INTO err\$_table</code> (with <code>DBMS_ERRLOG.CREATE_ERROR_LOG</code>)	a SQL-level way to let a big <code>INSERT / UPDATE / MERGE</code> skip failing rows and record them in a table
<code>EXCEPTION_INIT</code> on your <code>-20xxx</code> codes	give names to your application errors so callers can catch them by name
Cursor loops	<code>NO_DATA_FOUND</code> isn't raised by <code>FETCH</code> — use <code>%NOTFOUND</code> ; <code>INVALID_CURSOR</code> / <code>CURSOR_ALREADY_OPEN</code> come from mismanaged open/close

E6.14 Best practices

1. **Handle only what you can meaningfully handle**; let the rest propagate.
2. **Catch specific exceptions** first (`NO_DATA_FOUND` , `DUP_VAL_ON_INDEX`) and use `WHEN OTHERS` only last.
3. **Never** `WHEN OTHERS THEN NULL` . If you catch everything, **log it and re-raise** (`RAISE` ;).
4. **Validate early** (raise a clear error before doing work) rather than relying on the database to fail late.
5. **Use the sub-block pattern** to continue processing after a bad row.
6. **Give application errors names and constants** — keep the `-20xxx` numbers in a package so callers can `EXCEPTION_INIT` them.
7. **Keep messages useful**: include the key values (which employee id, which amount).
8. **Decide the transaction outcome** — after handling, explicitly `ROLLBACK` / `ROLLBACK TO SAVEPOINT` if partial work must be undone.
9. **Don't use exceptions for normal control flow** — check `SQL%ROWCOUNT` or use `IF` when "no row" is an expected case.

E6.15 The same idea in other systems

	Oracle PL/SQL	SQL Server T-SQL	PostgreSQL PL/pgSQL	Java-like languages
Structure	<code>BEGIN ... EXCEPTION WHEN ... THEN ... END;</code>	<code>BEGIN TRY ... END TRY BEGIN CATCH ... END CATCH</code>	<code>BEGIN ... EXCEPTION WHEN ... THEN ... END;</code>	<code>try { } catch { }</code>
Raise	<code>RAISE / RAISE_APPLICATION_ERROR</code>	<code>THROW / RAISERROR</code>	<code>RAISE EXCEPTION</code>	<code>throw</code>
Error info	<code>SQLCODE , SQLERRM</code>	<code>ERROR_NUMBER() , ERROR_MESSAGE()</code>	<code>SQLSTATE , SQLERRM</code>	exception object
Re-throw	<code>RAISE;</code>	<code>THROW;</code>	<code>RAISE;</code>	<code>throw e;</code>

Interview corner

Q. What is an exception in PL/SQL and how is it handled? A run-time error condition. When it is raised, execution jumps to the `EXCEPTION` section, where `WHEN ... THEN` handlers deal with it; unhandled exceptions propagate to the enclosing block or the caller.

Q. Types of exceptions? Predefined (named by Oracle, e.g. `NO_DATA_FOUND`), non-predefined (an Oracle error number you name with `PRAGMA EXCEPTION_INIT`), and user-defined (declared with `EXCEPTION` and raised with `RAISE`).

Q. What is the difference between `NO_DATA_FOUND` and `TOO_MANY_ROWS`? Both come from `SELECT ... INTO`: zero rows vs more than one row.

Q. `SQLCODE` vs `SQLERRM`? The number (negative, `+100` for no data found, `+1` for user-defined) and the message text of the current exception; they can't be used directly in SQL statements.

Q. `RAISE` vs `RAISE_APPLICATION_ERROR`? `RAISE` throws a named exception (its own number and message); `RAISE_APPLICATION_ERROR(-20000...-20999, msg)` throws an application-defined error with your own number and text to the caller.

Q. What is `PRAGMA EXCEPTION_INIT`? A compiler directive that associates a user-declared exception name with an Oracle error number so that error can be caught by name.

Q. What is the danger of `WHEN OTHERS`? It can hide real errors; use it only as a last handler that logs and re-raises.

Q. Does the block continue after an exception is handled? No — the block ends after the handler; control returns to the enclosing block. To continue a loop, wrap each iteration in its own `BEGIN ... EXCEPTION ... END`.

Q. What if an exception occurs in the `DECLARE` section? The same block's handler doesn't catch it; it propagates to the enclosing block.

Q. What happens to uncommitted changes when an exception is unhandled? When handled? Unhandled at the top level: the failed call is rolled back as a unit. Handled: nothing is rolled back automatically — you must `ROLLBACK` yourself.

Q. How do you log errors so that they survive a rollback? Log through a procedure declared with `PRAGMA AUTONOMOUS_TRANSACTION`, which commits independently.

Q. Which exceptions can a `SELECT MAX(x) INTO ...` raise when there are no matching rows? None — an aggregate without `GROUP BY` returns one row with `NULL`.

Quick recap

- `EXCEPTION WHEN name THEN ... WHEN OTHERS THEN ...` — the handler runs, then the **block ends**; `OTHERS` is last.
- Predefined: `NO_DATA_FOUND`, `TOO_MANY_ROWS`, `ZERO_DIVIDE`, `DUP_VAL_ON_INDEX`, `VALUE_ERROR`, `INVALID_NUMBER`, cursor and collection errors.
- `SQLCODE` (number; `+100` no data; `+1` user-defined) and `SQLERRM` (text) inside a handler — assign them to variables first.
- **User-defined:** `DECLARE e EXCEPTION;` → `RAISE e;` → `WHEN e THEN`. **Name an Oracle error:** `PRAGMA EXCEPTION_INIT(e, -2292)`.
- `RAISE_APPLICATION_ERROR(-20000...-20999, msg)` reports your own error to the caller; in a trigger it aborts the DML.
- Unhandled exceptions **propagate outward**; errors in `DECLARE` or inside a handler are *not* caught by the same block; bare `RAISE;` re-raises.
- Unhandled at the top level → the failed call is rolled back; handled → **no automatic rollback**. Log with `AUTONOMOUS_TRANSACTION`; never `WHEN OTHERS THEN NULL`.